CivicSuite - User Manual

This is the orientation manual for the CivicSuite umbrella repo. It is written in three parts so different audiences can read only what is relevant to them:

1. **For municipal decision-makers** - a non-technical overview of what CivicSuite is, what is available today, and what the sovereignty posture means for your city. 2. **For developers and IT staff** - how the umbrella repo works, where each module lives, and how to evaluate or contribute. 3. **Architecture reference** - the dependency rules, upgrade model, and suite topology.

A glossary at the end defines the technical terms used here.

Part 1 - For municipal decision-makers

What is CivicSuite?

CivicSuite is an **open-source municipal product family**. It is not one giant program. It is a planned collection of modules that a city can install one at a time, on its own hardware, on its own schedule. Cities do not need to send operational data to a vendor cloud, do not pay per seat, and can inspect or modify the source code at any time.

The suite is intentionally honest about maturity:

- `civicrecords-ai` is the current production-usable shipping product. - `civicclerk` is the active second-product candidate. - The rest of the catalog is in the foundation/planned tier: real runtime work or implementation specs, not yet end-to-end products. - `civiccore` is the shared platform package under all of them.

What is available today? (as of 2026-05-02)

- **`civicrecords-ai v1.4.5`** - the shipping product for public-records and FOIA workflow. Repo: <https://github.com/CivicSuite/civicrecords-ai> - **`civiccore v0.18.1`** - the shared platform package. It currently ships migrations, the shared SQLAlchemy `Base`, the LLM abstraction layer, audit/provenance primitives, export/manifest helpers, city profiles, auth/RBAC helpers, notice-compliance helpers, onboarding profile helpers, search/access helpers, connector/import helpers, live-sync retry/circuit primitives, release-evidence helpers, and trusted-header config/proxy enforcement helpers. Repo: <https://github.com/CivicSuite/civiccore> - **`civicclerk v0.1.15`** - the productizing second-product candidate for meetings, agendas, packets, minutes, voting, and sunshine-law compliance. It now ships all four MVP workflow surfaces in React, the resident public portal, Docker Compose product rehearsal, seeded Brookfield demo data, OIDC browser-session foundations, backup/restore rehearsal, vendor-network live sync with shared CivicCore retry/circuit primitives, scheduled local connector import sync, installer source packaging, enterprise signing readiness, and shared `civiccore v0.18.1` reuse. Repo: <https://github.com/CivicSuite/civicclerk> - **`CivicRegWatch` and `CivicAPI`** - newly added planned modules. CivicRegWatch is the federal regulatory intelligence module. CivicAPI is the public read-only data gateway over human-approved CivicSuite publication records. Their implementation specs live in specs/05_civicregwatch.md and specs/06_civicapi.md.

Selected foundation modules have also advanced beyond the original `civiccore==0.3.0` baseline. The authoritative truth for each module-to-platform pairing lives in the umbrella compatibility matrix, not in static prose snapshots:

- Compatibility matrix: docs/compatibility/index.md

What this means for your city

- **No vendor lock-in.** You run the software on your own hardware. If a maintainer disappears, your city still has the code and the right to keep using it. - **No mandatory cloud dependency.** CivicSuite is designed around local-first and sovereign deployment. - **No per-seat billing.** The licensing model does not meter users. - **Evaluate one module at a time.** A city does not need to adopt the whole suite at once. - **You still own operations.** This is not a managed SaaS product. Your IT team, or a contractor you choose, is responsible for installation, upgrades, and recovery.

The current suite tiers

| Tier | Count | Meaning today | |---|---:|---| | Shipping | 1 of 28 product modules | `civicrecords-ai` is the current production-usable module. | | Productizing | 1 of 28 product modules | `civicclerk` has real React product depth, Docker Compose product rehearsal, install rehearsal, backup/restore rehearsal, release handoff, OIDC browser-session foundations, vendor-network live sync, installer source packaging, and enterprise signing readiness. It still needs production deployment hardening before production city use. | | Foundation / planned | 26 of 28 product modules | The rest of the catalog has real runtime foundations or new implementation specs, but not yet full product depth. |

During the developer process, Windows installers should be treated as unsigned unless a module explicitly says otherwise. A first install can show a Windows SmartScreen or untrusted-publisher warning because code-signing certificates are not available for the whole developer cycle.

Foundation-tier module catalog

The rest of the catalog exists as real repositories with released runtime foundations. Their exact current versions and `civiccore` pairings should always be checked in:

- docs/compatibility/index.md - specs/01_catalog.md

Each foundation-tier module is explicit about what is shipped and what is still planned. The right mental model is "real foundations, not yet full products."

Glossary (Part 1)

- **FOIA** - Freedom of Information Act and related public-records laws. - **LLM** - Large language model. - **Open source** - software whose code is publicly available under a license that allows use, modification, and redistribution. - **Sovereign deployment** - software that runs on infrastructure the city controls.

Part 2 - For developers and IT staff

What lives in the umbrella repo?

The `civicsuite` repo is **documentation-first and coordination-first**. It contains:

- the [Charter](CHARTER.md) - the [Continuity plan](SUCCESSION.md) - the [Compatibility matrix](docs/compatibility/index.md) - the [Roadmap](docs/roadmap/index.md) - the [Shared extraction consumer rollout playbook](docs/roadmap/shared-extraction-consumer-rollout.md) - the unified suite specification and module catalog under `docs/` and `specs/` - suite-level governance and ADRs

It does **not** contain the runtime code for the individual products.

Module repos

| Repo | Status | |---|---| | `civicrecords-ai` | Shipping `v1.4.5` flagship product | | `civiccore` | Shipping `v0.18.1` shared platform package | | `civicclerk` | Productizing `v0.1.15` second-product candidate | | `civicregwatch` | Planned module; spec exists, repo not scaffolded yet | | `civicapi` | Planned module; spec exists, repo not scaffolded yet | | Remaining catalog repos | Foundation-tier runtime releases with bounded shipped surfaces |

Canonical GitHub locations:

- CivicSuite org: <https://github.com/CivicSuite> - `civicrecords-ai`: <https://github.com/CivicSuite/civicrecords-ai> - `civiccore`: <https://github.com/CivicSuite/civiccore> - `civicclerk`: <https://github.com/CivicSuite/civicclerk>

Dependency rule

The most important architectural rule in the suite is:

Modules depend on `civiccore`. `civiccore` never depends on modules.

That rule prevents hidden coupling between modules and preserves the promise that cities can deploy products independently.

How `civiccore` is consumed

Modules consume `civiccore` as a released dependency. The exact form depends on the release state:

- a published package version when available - a GitHub release wheel when that is the suite's current distribution path

The exact module-to-platform pairing is tracked in:

- docs/compatibility/index.md

Do not rely on static prose in old documents for version truth when the compatibility matrix is available.

How to evaluate a module

1. Read the module's `README.md`. 2. Read the module's `USER-MANUAL.md`. 3. Read the module's `CHANGELOG.md`. 4. Follow the module's `CONTRIBUTING.md` install steps on a clean machine. 5. Run the module's verification and test gates.

How releases are coordinated

When shared-platform behavior changes:

1. `civiccore` ships first. 2. Consumer modules adopt the new capability through a bounded rollout. 3. The compatibility matrix is updated. 4. Current-facing docs are updated in both the consumer repo and the umbrella repo when suite-level status changes.

The current standardized consumer adoption process lives here:

- docs/roadmap/shared-extraction-consumer-rollout.md

How to contribute

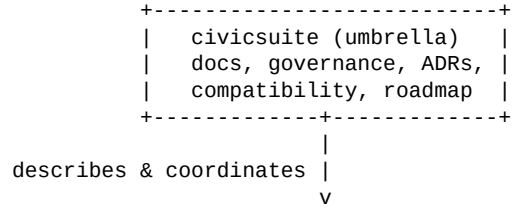
- Suite-wide roadmap, governance, compatibility, or umbrella documentation work belongs in this repo. - Product/module bugs and features belong in the relevant module repo.

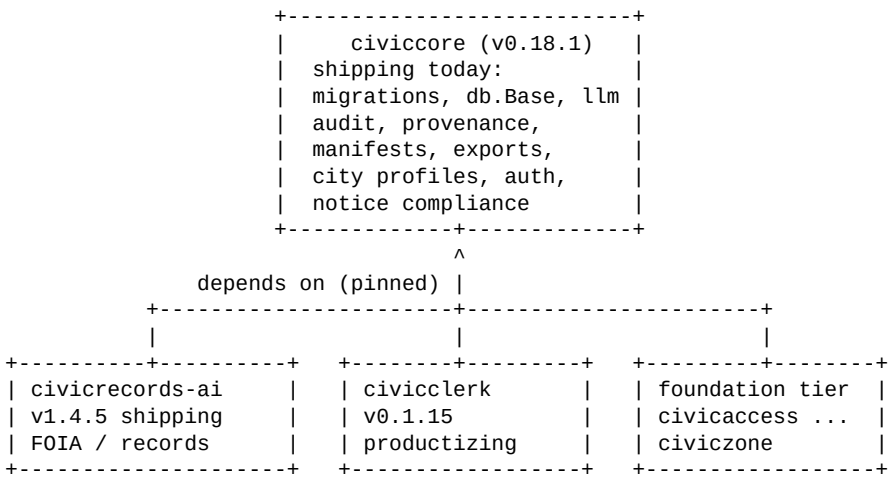
Start with:

- CONTRIBUTING.md - docs/governance/index.md

Part 3 - Architecture reference

Suite topology





Upgrade and migration order

When `civiccore` ships a backward-compatible change:

1. `civiccore` releases the new version. 2. The compatibility matrix is updated. 3. Consumer modules adopt the new version using the standard rollout playbook.

When `civiccore` ships a breaking change:

1. The change is documented in advance. 2. `civiccore` releases first. 3. Consumers ship paired changes. 4. The compatibility matrix records the new pairing.

Continuity and governance

Continuity is now a gate, not a future aspiration. The current continuity baseline is documented in:

- SUCCESSION.md

The roadmap that governs the rest of the program lives here:

- docs/roadmap/index.md

Glossary (Part 3)

- **ADR** - Architecture Decision Record. - **CI** - continuous integration. - **Pinned version** - a specific exact dependency pairing rather than a range. - **Wheel** - the Python package distribution format used for released artifacts. - **Monorepo** - a single repository containing multiple projects. CivicSuite is deliberately not a monorepo.

When something goes wrong

| Symptom | Where to look | |---|---| | Module will not install | The module repo's `README.md` and `CONTRIBUTING.md` | | `civiccore` version mismatch | docs/compatibility/index.md | | Unsure where to file a bug | CONTRIBUTING.md | | Security issue | SECURITY.md | | General support question | SUPPORT.md |